Traffic Light: Machine Coding Walkthrough

- **Problem:** Build a traffic light that cycles red → green → yellow → red forever, each light with its own duration.
- **Tech stack:** React + TypeScript (Vite)
- **Time limit:** 60 minutes (none was given, so the default is used)

PART A: Thought process

General steps for any machine coding task:

1. **Understand the requirements.** Restate the problem, split functional and non-functional needs, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action or event and what the UI must do. This becomes your checklist.
3. **Sketch the architecture.** Components, hooks, helpers, one responsibility each.
4. **Design the state.** Store the minimum, derive the rest, decide who owns what.
5. **Define data and contracts.** Sample data, props, hook signatures, API shapes.
6. **Find the hard parts early.** Timers, async, race conditions, accessibility.
7. **Build in layers.** A working ugly version first, then one working layer at a time.
8. **Cover states and edge cases.** Initial, running, paused, unmounted, invalid input.
9. **Wrap up.** Walk through trade-offs and what you would add with more time.

Time budget for 60 minutes:

Minutes	What
0-5	Requirements, clarifying questions, assumptions
5-10	Use cases, architecture, state model
10-30	Core: static lamps, config, timer cycle with cleanup
30-45	Hook extraction, pause/resume, status text, props
45-55	Edge cases, accessibility, styling
55-60	Demo, trade-offs, what you would add next

Tip: think out loud. The interviewer grades how you decide, not only the result. Say "I'll use one `setTimeout` per light instead of `setInterval` because each light has a different duration" before you type it. If you get stuck, say what you are trying, so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Show three lamps (red, yellow, green) stacked vertically.
- Exactly one lamp is on at any time; the others are dimmed.
- Cycle automatically: red (4 s) → green (3 s) → yellow (0.5 s) → red, forever.
- Durations and order come from a config, not hard-coded `if` chains.
- Show the current state as text ("RED: Stop").
- Extra (small, common follow-up): Pause / Resume button.

Non-functional requirements

Area	What it means here
Performance	One active timer at a time, no leaks, no re-render storms
Correctness	Each light holds for its own duration; no skipped or doubled lights in StrictMode
Error handling	Config is type-checked so `next` always points at a real light
Accessibility	Colour is never the only signal: visible status text, `role="status"` live region, lamps `aria-hidden`, reduced-motion respected
Reusability	`<TrafficLight config initial layout />`, timer logic in a hook
Extensibility	New light, new order or new durations = config change only

Clarifying questions to ask

1. What is the order and duration of each light? (Assume red 4 s, green 3 s, yellow 0.5 s.)
2. Which light should it start on?
3. Do you want manual controls: pause, skip, reset?
4. Should it show a countdown of the remaining time?
5. One light, or an intersection with several lights that must stay in sync?
6. Vertical only, or should horizontal layout be supported?

Assumptions we build with

- Starts on red; red 4000 ms, green 3000 ms, yellow 500 ms.
- A single traffic light.
- Pause/Resume is included; resuming restarts the current light's full duration (simpler; the "remaining time" version is a follow-up).
- No countdown display.
- Vertical by default, `layout="horizontal"` supported through CSS only.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads	Red lamp on, others dim, status "RED: Stop"
4 s pass on red	Green lamp on, status "GREEN: Go"
3 s pass on green	Yellow lamp on, status "YELLOW: Get ready to stop"
0.5 s pass on yellow	Red lamp on again, loop continues forever
Click **Pause**	Current lamp stays on, status gets "(paused)", button reads **Resume**
Click **Resume**	Cycle continues from the current light
Component unmounts	Pending timer is cleared, no state update after unmount
Custom `config` prop	Uses its durations, labels and order
Screen reader user	Each change is announced through the live region
`prefers-reduced-motion: reduce`	Lamps switch instantly, no glow transition

This table is the implementation and test checklist.

3. Design the architecture

Piece	Responsibility
`LIGHTS` (constants/lights.ts)	The state machine as data: duration, next light, label for each colour
`useTrafficLight` (hook)	Owns `active` and `paused`, runs the timer, cleans it up
`TrafficLight` (component)	Renders lamps, status text, pause button from the hook's output
`App`	Page shell, mounts one `TrafficLight`

```
App
└── TrafficLight            (UI: lamps + status + button)
    └── useTrafficLight     (logic: active, paused, timer)
            ▲
            └── LIGHTS      (config: duration / next / label)
```

Start simple: the whole thing can live in `App` for the first 15 minutes. Extract the hook once the timer works, because that is when its responsibility is clear. A separate `Lamp` component is not needed: a lamp is one `<span>` with no logic.

4. Design the state / data model

State or field	Purpose
`active: LightColor`	Which light is currently on
`paused: boolean`	Whether the cycle is frozen

```
const [active, setActive] = useState<LightColor>(initial)
const [paused, setPaused] = useState(false)
```

Senior note: derive, don't store

- Which lamp is on: `color === active`, no `isOn` per lamp.
- Current duration, next light, label: `config[active]`.
- The timer id lives in the effect's closure, not in state (it never affects rendering).
- **Ownership:** the hook owns `active` and `paused`. The config is owned by the caller (a module-level constant, so its identity is stable).

5. Design the data structures and contracts

Config: the state machine as data

```
export const LIGHTS: Record<LightColor, LightConfig> = {
  red:    { duration: 4000, next: 'green',  label: 'Stop' },
  yellow: { duration: 500,  next: 'red',    label: 'Get ready to stop' },
  green:  { duration: 3000, next: 'yellow', label: 'Go' },
}
```

Why this shape:

- The colour is the stable id (also the React `key`), so no generated ids are needed.
- `next` is an explicit pointer, so the cycle order (red → green → yellow) is separate from the render order (red, yellow, green top to bottom). An array with `index + 1` would force them to be the same.
- `Record<LightColor, …>` makes TypeScript reject a missing light or a typo in `next`.

Component contract

```
type Props = {
  config?: Record<LightColor, LightConfig> // default LIGHTS
  initial?: LightColor                     // default 'red'
  layout?: 'vertical' | 'horizontal'       // default 'vertical'
}
```

Hook contract

```
useTrafficLight(config, initial) => { active: LightColor; paused: boolean; togglePause: () => void
}
```

There is no API. If the config came from a server, the payload would be the same JSON shape, validated at the boundary (every `next` exists, every `duration > 0`) before use:

```
{ "red": { "duration": 4000, "next": "green", "label": "Stop" }, "...": "..." }
```

6. Trace the data flow

Key interaction: red turns green.

```
mount
  │  active = 'red'
  ▼
useEffect([active, paused, config])
  │  setTimeout(..., config.red.duration = 4000)
  ▼
4000 ms later: timeout fires
  │  setActive(config.red.next)  → active = 'green'
  ▼
re-render
  │  green <span> gets class "on", red loses it
  │  status text → "GREEN: Go"  (live region announces it)
  ▼
effect cleanup (clearTimeout of the old id) → effect runs again
  │  setTimeout(..., config.green.duration = 3000)
  ▼
… loop continues forever
```

7. Important technical challenges

Challenge 1: Every light has a different duration

Problem: `setInterval(next, 4000)` ticks at a fixed rate, so yellow would stay on for 4 s instead of 0.5 s.

Solution: schedule one `setTimeout` per light. The effect depends on `active`, so every change of light schedules the next timeout with that light's own duration.

```
useEffect(() => {
  const id = setTimeout(() => setActive(config[active].next), config[active].duration)
  return () => clearTimeout(id)
}, [active, config])
```

Challenge 2: Timer cleanup and React StrictMode

Problem: in development StrictMode mounts, unmounts and re-mounts every component. Without cleanup two timeouts run, the light changes twice and skips states. In production, unmounting mid-cycle leaves a timer that calls `setActive` on a dead component.

Solution: always return `clearTimeout` from the effect. Cleanup also runs before every re-run, so there is never more than one pending timer.

```
return () => clearTimeout(id)
```

Challenge 3: Stale closures

Problem: a timer created once with `[]` dependencies captures `active = 'red'` forever:

```
useEffect(() => {
  setInterval(() => setActive(config[active].next), 1000) // active is always 'red'
}, [])
// result: red → green → green → green … stuck
```

Solution: either list `active` as a dependency (what we do, it also gives per-light durations), or use the functional updater `setActive(a => config[a].next)` which always reads the latest value.

Challenge 4: Pause/resume and config identity

Problem 1: pausing must stop the timer without losing the current light. **Problem 2:** if a parent writes `<TrafficLight config={{ ... }} />`, a new object is created on every parent render. `config` is an effect dependency, so every parent re-render restarts the timer, and a parent that re-renders every second would freeze the light.

Solution: `paused` is a dependency; when true the effect returns early (and the previous run's cleanup already cleared the timer). Keep the config a module-level constant, or `useMemo` it in the parent.

```
useEffect(() => {
  if (paused) return
  const id = setTimeout(() => setActive(config[active].next), config[active].duration)
  return () => clearTimeout(id)
}, [active, paused, config])
```

Trade-off: resume restarts the current light's full duration. To resume with the remaining time, store `startedAt` and `remaining` in refs (see the counter questions).

8. Implement in the correct order

#	Step	Working result
1	Clean the Vite template, render a heading	App runs
2	Static markup: three `<span>` lamps + CSS (dim, `.on` glows) with red hard-coded on	Looks like a traffic light
3	Add `LightColor`, `LightConfig` types and the `LIGHTS` config	Data drives the lamps
4	`active` state; `.on` from `color === active`	Changing state changes the lamp
5	`useEffect` + `setTimeout` to `config[active].next`, with cleanup	Full automatic loop
6	Check the loop in StrictMode, check timing of each light	No skipped lights
7	Extract `useTrafficLight(config, initial)`	Logic reusable, component is just UI
8	Status text in a `role="status"` live region, lamps `aria-hidden`	Accessible, not colour-only
9	`paused` state + Pause/Resume button	Freeze and continue
10	Props `config`, `initial`, `layout` with defaults	Reusable component
11	Reduced-motion CSS, horizontal layout, polish	Done
12	Walk through edge cases and trade-offs	Interview wrap-up

9. Cover all states and edge cases

State / edge case	Behavior
Initial	`initial` light (red) on, status shows it
Running	One timeout pending for the current light
Paused	No timeout pending, status shows "(paused)", button says Resume
Resumed	New timeout with the current light's full duration
Unmounted	Cleanup clears the timeout, no state update after unmount
StrictMode double mount	First effect's cleanup clears its timeout, only one timer remains
Parent re-renders with the same config	Timer is not reset (stable dependency)
Parent passes a new config object each render	Timer resets each render: documented, fixed with a constant or `useMemo`
`initial` prop changes after mount	Ignored (it only seeds `useState`); remount with a `key` to reset
`next` points at a missing light	Rejected at compile time by `Record<LightColor, …>`
Duration 0	Light flashes for one frame; acceptable, or validate `> 0` at the boundary
Tab in background	Browser throttles timers; light catches up when visible (fine for a visual)
Colour-blind or screen-reader user	Visible text + live region announce every change
Reduced motion	No transition on lamps

10. Final structure

```
_30_trafic_light/
└── src/
    ├── constants/
    │   └── lights.ts           # LightColor, LightConfig, LIGHTS
    ├── hooks/
    │   └── useTrafficLight.ts  # active, paused, timer + cleanup
    ├── components/
    │   └── TrafficLight.tsx    # lamps, status, pause button
    ├── App.tsx                 # page shell
    ├── App.css                 # lamp + layout styles
    ├── index.css               # Vite base styles (unchanged)
    └── main.tsx                # entry (unchanged)
```

11. Full working code

src/constants/lights.ts

```
export type LightColor = 'red' | 'yellow' | 'green'

export type LightConfig = {
  duration: number // ms this light stays on
  next: LightColor
  label: string // spoken/visible text, so colour is never the only signal
}

// Key order = top-to-bottom render order of the lamps
export const LIGHTS: Record<LightColor, LightConfig> = {
  red: { duration: 4000, next: 'green', label: 'Stop' },
  yellow: { duration: 500, next: 'red', label: 'Get ready to stop' },
  green: { duration: 3000, next: 'yellow', label: 'Go' },
}
```

src/hooks/useTrafficLight.ts

```
import { useEffect, useState } from 'react'
import type { LightColor, LightConfig } from '../constants/lights'

export function useTrafficLight(
  config: Record<LightColor, LightConfig>,
  initial: LightColor,
) {
  const [active, setActive] = useState<LightColor>(initial)
  const [paused, setPaused] = useState(false)

  useEffect(() => {
    if (paused) return
    // One timeout per light (not setInterval): every light has its own duration
    const id = setTimeout(
      () => setActive(config[active].next),
      config[active].duration,
    )
    return () => clearTimeout(id)
  }, [active, paused, config])

  return { active, paused, togglePause: () => setPaused((p) => !p) }
}
```

src/components/TrafficLight.tsx

```
import { LIGHTS, type LightColor, type LightConfig } from '../constants/lights'
import { useTrafficLight } from '../hooks/useTrafficLight'

type Props = {
  config?: Record<LightColor, LightConfig>
  initial?: LightColor
  layout?: 'vertical' | 'horizontal'
}

export default function TrafficLight({
  config = LIGHTS,
  initial = 'red',
  layout = 'vertical',
}: Props) {
  const { active, paused, togglePause } = useTrafficLight(config, initial)

  return (
    <div className="traffic-light-wrapper">
      <div className={`traffic-light ${layout}`} aria-hidden="true">
        {(Object.keys(config) as LightColor[]).map((color) => (
          <span
            key={color}
            className={`lamp ${color}${color === active ? ' on' : ''}`}
          />
        ))}
      </div>

      <p className="status" role="status" aria-live="polite">
        {active.toUpperCase()}: {config[active].label}
        {paused && ' (paused)'}
      </p>

      <button type="button" onClick={togglePause}>
        {paused ? 'Resume' : 'Pause'}
      </button>
    </div>
  )
}
```

src/App.tsx

```
import TrafficLight from './components/TrafficLight'
import './App.css'

function App() {
  return (
    <main>
      <h1>Traffic Light</h1>
      <TrafficLight />
    </main>
  )
}

export default App
```

src/App.css

```
#root {
  margin: 0 auto;
  padding: 2rem;
  text-align: center;
}

.traffic-light-wrapper {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 1rem;
}

.traffic-light {
  display: flex;
  flex-direction: column;
  gap: 12px;
  padding: 16px;
  background: #111;
  border: 2px solid #444;
  border-radius: 16px;
}

.traffic-light.horizontal {
  flex-direction: row;
}

.lamp {
  width: 72px;
  height: 72px;
  border-radius: 50%;
  background: var(--lamp);
  opacity: 0.2;
  transition: opacity 200ms, box-shadow 200ms;
}

.lamp.red { --lamp: #e53935; }
.lamp.yellow { --lamp: #fdd835; }
.lamp.green { --lamp: #43a047; }

.lamp.on {
  opacity: 1;
  box-shadow: 0 0 24px var(--lamp);
}
```

```
.status {
  margin: 0;
  font-weight: 600;
}

@media (prefers-reduced-motion: reduce) {
  .lamp {
    transition: none;
  }
}
```

`main.tsx` and `index.css` stay as the Vite template generated them.

12. Wrap-up

Edge cases handled

- Different duration per light, infinite loop.
- Single pending timer; cleanup on unmount, on pause and in StrictMode.
- No stale closures: the effect re-runs on every light change.
- Type-safe config: no missing light, no typo in `next`.
- Accessible: text status in a live region, decorative lamps hidden, reduced motion.

Trade-offs to mention

- **`setTimeout` per light vs one `setInterval` tick:** per-light timeouts are simple and exact per state; a 1 s tick would be needed for a countdown but makes 0.5 s yellow awkward.
- **Config as `Record` with `next` vs array with index:** decouples render order from cycle order and allows non-linear sequences, at the cost of a tiny bit more data.
- **`useState` vs `useReducer`:** two independent values are fine with `useState`; once there are events like SKIP, RESET, EMERGENCY, a reducer (an explicit state machine) is clearer.
- **Pause restarts the full duration:** simpler; exact resume needs `startedAt` and `remaining` refs.
- **Chained timeouts drift a few ms per cycle:** irrelevant for a UI light; a real controller would compute the phase from an absolute start time.

With more time

- Tests with Vitest fake timers and React Testing Library:

```
vi.useFakeTimers()
render(<TrafficLight />)
expect(screen.getByRole('status')).toHaveTextContent('RED')
act(() => vi.advanceTimersByTime(4000))
expect(screen.getByRole('status')).toHaveTextContent('GREEN')
act(() => vi.advanceTimersByTime(3000))
expect(screen.getByRole('status')).toHaveTextContent('YELLOW')
```

- Countdown of remaining seconds; exact pause/resume.

- Intersection controller: one state machine driving north-south and east-west lights.
- Night mode (flashing yellow), pedestrian request button, emergency override.

13. Counter questions the interviewer may ask

Requirements

Q1. Why did you ask about the order and durations instead of just picking them?

Because they decide the data model. If every light had the same duration, a `setInterval` would do; different durations force a per-state timeout. If the order were not a simple loop (e.g. UK red+amber), an array index would not work. One question saves a rewrite.

Q2. What if the interviewer adds a red+amber phase, where two lamps are on at once?

Today one phase = one colour. Change the config to phases that list the lamps they light: `{ id: 'redAmber', on: ['red', 'yellow'], duration: 2000, next: 'green' }`, keep `active` as the phase id, and render `.on` when `phase.on.includes(color)`. The timer hook does not change.

State design

Q3. Why store only `active` and not an `isOn` flag for each lamp?

Three flags can disagree (two lamps on at once). `active` is the single source of truth and `color === active` is derived. Fewer states, impossible states made impossible.

Q4. Why is the timer id not in state or a ref?

It never affects rendering, so state would cause a useless re-render. It is only needed by the cleanup of the same effect run, so the effect's closure is enough. A ref is needed only if something outside the effect (e.g. a Skip button) must clear it.

Q5. Why `useState` and not `useReducer` or a state machine library?

Two independent values and two transitions (timer, toggle pause) are simple. When events grow (SKIP, RESET, EMERGENCY, NIGHT_MODE), I'd move to `useReducer` with an explicit `transition(state, event)` function, which is also easy to unit test. XState is overkill for a 60-minute round.

Hard challenge: timers

Q6. Why not `setInterval`?

It fires at a fixed rate, but each light has a different duration. You'd need a fast tick (e.g. 100 ms) plus an elapsed counter, which is more state and more renders. One `setTimeout` per light does exactly what the spec says.

Q7. What happens in React StrictMode in development?

The effect runs, its cleanup runs, then it runs again. Because cleanup clears the first timeout, only one timer survives. Without the cleanup the light would change twice per period and skip states, which is a classic bug interviewers look for.

Q8. What is a stale closure, and where could it bite here?

A callback that captured an old value. `setInterval(() => setActive(config[active].next), …)` created once with `[]` deps always sees `active = 'red'`, so the light gets stuck on green. Fix: put `active` in the deps (our approach) or use `setActive(a => config[a].next)`.

Q9. What if the parent passes `config={{ ... }}` inline?

A new object every render changes the effect's dependency, so the timer restarts on every parent render; if the parent re-renders faster than the light's duration, the light never changes. Fix: module constant or `useMemo` in the parent. Defensive alternative: keep config in a ref inside the hook and depend only on `active`.

Q10. How would you resume with the remaining time instead of restarting?

Store `startedAt` (when the current timeout was scheduled) and `remaining` in refs. On pause: `remaining = remaining - (Date.now() - startedAt)` and clear the timeout. On resume: schedule `setTimeout(next, remaining)`. When the light changes, reset `remaining = config[active].duration`.

Q11. How would you show a countdown?

Add a 1 s (or `requestAnimationFrame`) tick that updates a `now` state, and derive `remaining = startedAt + duration - now`. Keep the light change on its own exact timeout so the countdown display never controls the logic.

Implementation steps

Q12. Why extract a hook instead of keeping everything in the component?

The timer is the logic; lamps and button are the view. The hook can be reused (e.g. a horizontal light, a pedestrian light) and tested alone with `renderHook` and fake timers. I extract it only after the timer works, not upfront.

Q13. Why `aria-hidden` on the lamps and a separate status text?

The lamps are decorative circles; a screen reader would read nothing useful from them. The `role="status"` paragraph is a polite live region, so each change ("GREEN: Go") is announced, and sighted colour-blind users read the same text.

Q14. How do you test this?

Vitest + React Testing Library with fake timers: render, assert RED, `advanceTimersByTime(4000)`, assert GREEN, and so on. Also: pause then advance 10 s and assert unchanged; unmount and assert no warnings; `vi.getTimerCount()` is 1 while running and 0 when paused.

Performance and "what if" twists

Q15. What if the page shows 1,000 traffic lights?

1,000 timers and 1,000 uncoordinated renders. Use one shared clock (a single `setInterval` or `requestAnimationFrame` in a context or store) and derive each light's phase from elapsed time: `t = (now -`

`start + offset) % cycleLength`, then find the phase whose cumulative duration covers `t`. Only lights whose phase changed re-render (memoised components).

Q16. What if the browser tab is in the background?

Browsers throttle timers in hidden tabs (often to 1 s or more). Our light just changes late; when the tab comes back it continues. If the phase must be correct after returning, compute it from absolute time (Q15) or re-sync on `visibilitychange`.

Q17. What if the config comes from a slow API?

Render with the default `LIGHTS` (or a loading state with all lamps off), validate the response at the boundary (every `next` exists, every `duration > 0`, fall back on error), then pass it in. Use a `key` to remount the light so it starts cleanly with the new config.

Q18. Do chained timeouts drift?

Yes, each `setTimeout` fires a few ms late and the error accumulates over hours. For a UI it doesn't matter. For a synced intersection, compute the phase from a fixed start time so errors don't accumulate.

Extensions and trade-offs

Q19. How would you build a 4-way intersection?

Lift state into one controller with one state machine; derive each direction: north-south green implies east-west red. Never run two independent timers that must stay in sync, they drift. Render four `TrafficLight` views as pure components that take `active` as a prop.

Q20. How would you add an emergency override (all red now)?

Add an event: with `useReducer`, dispatch `EMERGENCY` → state `{ mode: 'emergency' }`, render all red, the effect skips the timer in that mode; `CLEAR` resumes the normal cycle from red. This is the point where the reducer pays off.

Q21. What if `initial` changes after mount?

It is ignored because it only seeds `useState`. That is intended: the prop is a starting point, not a controlled value. To reset, the parent changes the component's `key`. For a fully controlled light, accept `active` and `onChange` props instead.